BVC ENGINEERING COLLEGE (AUTONOMOUS)

Approved by AICTE, New Delhi • Permanently Affiliated to JNTUK, Kakinada
Accredited by NAAC • Odalarevu, Dr. B.R. Ambedkar Konaseema District, A.P.

A TECHNICAL PROJECT REPORT ON

ACADEMIAQUIZ

B.Tech Online Quiz & Real-Time Multi-Device Leaderboard System

Featuring Zero Dummy Accounts, Cross-Device Cloud Sync & 8-Semester Curriculum

Submitted in partial fulfillment of the academic requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

Submitted by:
SRITEJA MALLULA
Department of Computer Science & Engineering
BVC Engineering College (Autonomous)

Under the Guidance of:
PROJECT GUIDE
Department of Computer Science & Engineering
BVC Engineering College (Autonomous)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ACADEMIC YEAR 2025 – 2026

BVC ENGINEERING COLLEGE (AUTONOMOUS)

Odalarevu, Dr. B.R. Ambedkar Konaseema District, Andhra Pradesh – 533210
Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the technical project report entitled **"ACADEMIAQUIZ: B.TECH ONLINE QUIZ & REAL-TIME MULTI-DEVICE LEADERBOARD SYSTEM"** is a bonafide work carried out and submitted by **SRITEJA MALLULA** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering** to **Jawaharlal Nehru Technological University Kakinada (JNTUK)**, during the academic year 2025 – 2026.

The results embodied in this report have been verified, tested on multiple mobile and desktop hardware clients, and have not been submitted to any other University or Institute for the award of any degree or diploma.

Project Guide
Assistant Professor / Assoc. Prof.
Dept. of CSE, BVCEC

Head of the Department
Department of CSE
BVCEC (Autonomous)

Internal Examiner
Date: _____

External Examiner
Date: _____

Declaration by the Candidate

I, **SRITEJA MALLULA**, hereby declare that the project report entitled **"ACADEMIAQUIZ: B.TECH ONLINE QUIZ & REAL-TIME MULTI-DEVICE LEADERBOARD SYSTEM"** submitted to the Department of Computer Science & Engineering, BVC Engineering College (Autonomous), Odalarevu, is a genuine record of original work done by me under the guidance of the faculty.

I further declare that this software application has been deployed live to the public internet at **https://sritejamallula.github.io/online-quiz/**, verified for cross-device real-time cloud data synchronization, and engineered without mock/dummy student accounts.

Place: Odalarevu
Date: October 20, 2026

SRITEJA MALLULA
Candidate, Dept of CSE

Acknowledgements

I express my deepest gratitude to the respected **Principal** and **Management** of BVC Engineering College (Autonomous), Odalarevu, for providing state-of-the-art laboratory and computing facilities to execute this academic endeavor.

I render my sincere thanks to the **Head of the Department (HOD)**, Department of Computer Science & Engineering, for their invaluable support, guidance, and continuous encouragement throughout the semester.

Special thanks are due to my **Project Guide** and faculty members of the CSE department whose technical insights on the R23 curriculum and algorithmic formulations were instrumental in refining this system. Finally, I thank my **classmates** whose participation across varied devices validated the real-time cross-device synchronization engine.

Abstract

Contemporary educational evaluation tools often rely on localized browser storage (`localStorage`), which limits candidate data to the specific device on which an assessment is taken. Consequently, classroom instructors must rely on pre-populated synthetic mock accounts to demonstrate leaderboard rankings. Furthermore, standard testing tools fail to match the specific unit-level theory curriculum prescribed by engineering universities.

AcademiaQuiz addresses these core limitations by presenting a modern, multi-device academic evaluation and live ranking platform developed specifically for B.Tech Computer Science & Engineering undergraduates under the BVCEC Autonomous R23 curriculum. The system guarantees a **Zero Dummy Users** policy: the database starts completely clean, and leaderboards are populated strictly by genuine classmates who access the platform and register.

A dedicated **Real-Time Cloud Synchronization Engine** connects clients to a centralized REST persistence endpoint. Quiz submissions, registrations, and scores propagate to all connected student smartphones, laptops, and the instructor portal within 8 seconds using multi-trigger event handlers (background interval polling, window focus listeners, and cross-tab `BroadcastChannel` communication).

The platform features an 8-semester catalog comprising **40 core theory engineering subjects**. Each subject includes 10-question evaluation quizzes equipped with timed countdowns, interactive question palette tracking grids, and post-submission GeeksforGeeks-style step-by-step technical explanations. Advanced algorithmic foundations are comprehensively integrated into both the curriculum evaluation engine (AVL Trees, Red-Black Trees, Bellman-Ford, Floyd-Warshall, 0/1 Knapsack, Matrix Chain Multiplication, Disjoint Set Union) and the platform itself (hybrid rank scoring, set-based attempt deduplication, and timeline promotion). The platform is deployed publicly on GitHub Pages.

Keywords: Real-Time Cloud Synchronization, B.Tech CSE Curriculum, Zero Dummy Users, Advanced Data Structures (ADS), Online Assessment Engine, Dynamic Leaderboard Podium, React 18, REST Persistence.

Table of Contents

Chapter	Title / Description	Page
—	Certificate of Authenticity	2
—	Declaration & Acknowledgements	3
—	Abstract	4
Chapter 1	Introduction, Background & Problem Formulation	6
Chapter 2	System Requirements & Specification Analysis	7
Chapter 3	System Architecture & Data Flow Design	8
Chapter 4	Algorithmic Foundations & Computational Analysis (ADS & Internal)	9
Chapter 5	Real-Time Cross-Device Cloud Synchronization Engine	11
Chapter 6	Curriculum Mapping & 40-Subject Engineering Matrix	12
Chapter 7	User Experience, Examination Engine & Faculty Portal	14
Chapter 8	Testing, Verification & Cross-Device Benchmarks	15
Chapter 9	Conclusion, Deployment & Future Enhancements	15
—	References & Technical Bibliography	16

Chapter 1: Introduction & Problem Formulation

1.1 Project Background

The computer science engineering education framework requires consistent assessment of both theoretical comprehension and algorithmic problem-solving abilities. In contemporary engineering colleges, traditional paper-based weekly quizzes incur high administrative overhead, delayed performance feedback, and lack peer visibility. While several commercial online quiz platforms exist, they fail to align with autonomous university syllabi, require subscription fees, or require cumbersome account onboarding.

1.2 Problem Statement

Traditional client-side web applications store student profiles and quiz attempts in the browser\'s local storage (`localStorage`). This architecture creates critical operational deficiencies:

1. **Device Data Siloing:** When a student takes a quiz on their smartphone, their score is inaccessible from a laptop or desktop computer.
2. **Need for Artificial / Dummy Data:** To showcase a functional leaderboard or faculty analytics, developers pre-populate mock students. These artificial accounts distort class performance statistics and diminish authentic peer motivation.
3. **Absence of Classroom Interoperability:** In a lab or classroom setting, an instructor cannot view an aggregate, live leaderboard of real student attempts without custom backend servers.

1.3 Objectives of the Project

- **Universal Real-Time Sync:** Build a zero-maintenance cloud persistence engine that synchronizes student registrations and quiz attempts across all client devices within an 8-second polling threshold.
- **Zero Dummy Accounts Policy:** Guarantee that no artificial or hardcoded user accounts exist, ensuring that 100% of participants on leaderboards and rosters are genuine classmates.
- **R23 Curriculum Coverage:** Formulate 40 comprehensive theory subject modules mapping to the 8 semesters of the BVCEC B.Tech CSE syllabus.
- **Pedagogical Explanations:** Provide step-by-step GeeksforGeeks-style rationale and code snippets for every question to support self-directed learning.

Chapter 2: System Requirements & Specification

2.1 Software & Hardware Requirements

Component	Specification Details
Client Hardware	Any standard Android/iOS smartphone, tablet, laptop, or desktop with 1 GB available RAM and internet connectivity.
Supported Browsers	Google Chrome (v90+), Microsoft Edge (v90+), Mozilla Firefox (v88+), Apple Safari (v14+).
Frontend Stack	React 18.3, TypeScript 5.5, Vite 6.4, Tailwind CSS 4.0.
UI & Animation Libraries	Lucide React (vector iconography), Canvas Confetti (celebratory triggers).
Cloud Persistence	RESTful API Object Store (`api.restful-api.dev`) with CORS wildcard support.
Production Hosting	GitHub Pages Static Server (HTTPS enforced, CDN distributed).

2.2 Functional Requirements

- **FR-1 (Registration):** Students register with their Full Name, Roll Number, Branch (`CSE`, `AI&DS`, `IT`, `ECE`, `EEE`), Admission Year, and Semester.
- **FR-2 (Cloud Propagation):** Newly registered profiles are dispatched via HTTPS PUT to the cloud within 1 second.
- **FR-3 (Timed Examination):** Quizzes enforce a 10-minute timer with real-time question palette tracking and automatic submission.
- **FR-4 (Pedagogical Evaluation):** Upon submission, students receive scores, percentages, and technical explanations.
- **FR-5 (Dynamic Leaderboard):** Ranks are computed dynamically from real student attempts using a hybrid formula.
- **FR-6 (Faculty Portal):** Instructors can view class KPIs, modify assessment cutoffs, and audit submission logs.

2.3 Non-Functional Requirements

- **Performance:** Initial application load time under 1.5 seconds over 4G mobile networks.
- **Zero Dummy Policy:** Strictly zero fallback mock users in production state.
- **Cross-Device Responsiveness:** Fluid UI layout adaptation across mobile (360px) to ultra-wide (2560px) screens.
- **Data Integrity:** Conflict-free de-duplication using roll number keys and UUIDv4 submission identifiers.

Chapter 3: System Architecture & Data Flow

3.1 Layered Architecture Overview

AcademiaQuiz adopts a 4-tier decoupled single-page application architecture:

Tier	Modules / Components	Design Rationale & Functional Scope
Presentation Tier	`Navbar`, `SubjectGrid`, `QuizEngine`, `LeaderboardTable`, `FacultyDashboard`	Crafted with Tailwind CSS 4 light glassmorphism (`bg-white/85`, `backdrop-blur-xl`, `border-slate-200`) providing high contrast slate typography and responsive touch layouts.
Application State	`QuizContext.tsx`	Centralizes registered student rosters, completed quiz attempt logs, active semester selection, timer thresholds, and authentication dialog states.
Synchronization Layer	`cloudSync.ts`, HTML5 `BroadcastChannel`	Handles asynchronous REST GET/PUT requests, 8-second polling scheduling, tab focus event listening, and conflict-free data de-duplication.
Persistence Layer	Cloud REST Database, Browser `localStorage`	Maintains the global JSON payload online while caching state locally to provide continuous offline test taking.

3.2 Execution Data Flow

1. **Student Entry:** The candidate selects a subject and clicks "Start Quiz". The system verifies active registration; if unregistered, an authentication dialog prompts for candidate credentials.
2. **State Dispatch:** Upon submission of candidate details, the `loginStudent()` action adds the profile to `registeredStudents` in React state and invokes `pushToCloud()`.
3. **Local & Cloud Synchronization:** The sync engine serializes the payload into `data.payload` and submits an HTTPS PUT request to the cloud endpoint. Concurrently, a message is dispatched to `"quiz_sync_channel"`.
4. **Quiz Lifecycle:** The 10-question evaluation engine initiates the 10-minute timer. Responses are stored in volatile component state.
5. **Score Submission:** Upon confirmation, `submitQuiz()` computes accuracy, records elapsed seconds, appends the attempt to `quizAttempts`, updates candidate metrics, and pushes the new dataset to the cloud.
6. **Global Propagation:** All other student devices and the faculty portal receive the updated record on their next 8-second polling cycle or immediately when the tab is focused.

Chapter 4: Algorithmic Foundations & Computational Analysis

Algorithms serve two critical roles in AcademiaQuiz: (1) evaluating students on core computer science algorithms in the **Advanced Data Structures (ADS — CS2104)** curriculum, and (2) powering the platform\'s real-time ranking, conflict resolution, and semester promotion engines.

4.1 Advanced Data Structures (ADS — CS2104) Curriculum Algorithms

Algorithm / Concept	Domain	Time / Space Complexity	Algorithmic Principle & Purpose
AVL Tree Balancing	Self-Balancing BST	Search: $O(\log N)$ Insert: $O(\log N)$ Space: $O(N)$	Maintains strict height balance: $\|Height(Left) - Height(Right)\| \le 1$. Employs Single (LL, RR) and Double (LR, RL) rotations.
Red-Black Tree Balancing	Self-Balancing BST	Search: $O(\log N)$ Insert: $O(\log N)$ Space: $O(N)$	Enforces 5 coloring rules (Root is black, No two consecutive red nodes, Equal black-height). Height is bounded by $2\log_2(N+1)$.
Bellman-Ford Algorithm	Graph Shortest Path	Time: $O(V \cdot E)$ Space: $O(V)$	Relaxes all E edges $V-1$ times. Capable of handling negative edge weights and detecting negative weight cycles.
Floyd-Warshall Algorithm	All-Pairs Shortest Path	Time: $O(V^3)$ Space: $O(V^2)$	Dynamic programming algorithm computing shortest paths between all pairs of vertices via 3 nested loops: $D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$.
0/1 Knapsack Problem	Dynamic Programming	Time: $O(N \cdot W)$ Space: $O(N \cdot W)$	Constructs a 2D table using optimal substructure: $DP[i][w] = \max(DP[i-1][w], DP[i-1][w-w_i] + v_i)$. Demonstrates pseudo-polynomial time complexity.
Matrix Chain Multiplication	Dynamic Programming	Time: $O(N^3)$ Space: $O(N^2)$	Finds optimal parenthesization of N matrices to minimize scalar multiplications using split indices: $m[i,j] = \min_{i \le k < j}(m[i,k] + m[k+1,j] + p_{i-1}p_k p_j)$.
Disjoint Set Union (DSU)	Disjoint Sets	Time: $O(\alpha(N))$ amortized Space: $O(N)$	Maintains partition sets. Achieves nearly $O(1)$ amortized operations by combining **Path Compression** and **Union by Rank**.
Master Theorem Analysis	Asymptotic Analysis	Recurrences: $T(N) = aT(N/b) + f(N)$	Solves divide-and-conquer recurrences. For $T(N) = 2T(N/2) + O(N)$, Case 2 applies since $N^{\log_2 2} = N$, giving $\Theta(N \log N)$.
NP-Completeness	Complexity Theory	Polynomial verification (NP)	Assesses non-deterministic polynomial-time complete problems (3-SAT and Decision Knapsack) via polynomial reductions.

4.2 Internal System Algorithms (Platform Engine)

4.2.1 Dynamic Leaderboard Hybrid Ranking Algorithm

To rank classmates accurately and eliminate ties among students with identical point totals, AcademiaQuiz employs a continuous hybrid scoring formula implemented in `QuizContext.tsx`:

```
function computeLeaderboard(students: StudentProfile[]): LeaderboardEntry[] {
 return students
   .filter(s => s.quizzesAttempted > 0)
   .map(student => {
     // Hybrid scoring formula combining raw points with weighted accuracy
     const rankScore = student.totalPoints + (student.averageScore * 0.1);
     return { student, rankScore };
   })
   .sort((a, b) => {
     // Primary: Weighted Rank Score descending
     if (b.rankScore !== a.rankScore) return b.rankScore - a.rankScore;
     // Secondary tie-breaker: Average accuracy percentage
     if (b.student.averageScore !== a.student.averageScore) {
       return b.student.averageScore - a.student.averageScore;
     }
     // Tertiary tie-breaker: Number of quizzes completed
     return b.student.quizzesAttempted - a.student.quizzesAttempted;
   })
   .map((entry, index) => ({
     rank: index + 1,
     student: entry.student,
     totalScore: entry.student.totalPoints,
     accuracy: entry.student.averageScore,
     quizzesTaken: entry.student.quizzesAttempted,
     badge: index === 0 ? \'gold\' : index === 1 ? \'silver\' : index === 2 ? \'bronze\' : undefined
   }));
}
```

4.2.2 Set-Based Conflict-Free De-duplication Algorithm

To merge cloud updates with local state without data corruption, `cloudSync.ts` implements two distinct conflict-free algorithms:

- **Student Roster Merge:** Profiles are keyed by normalized roll number. When both sources contain matching roll numbers, the algorithm compares ISO 8601 timestamps and preserves the most recent update:
  `selected = new Date(remote.registeredAt) > new Date(local.registeredAt) ? remote : local`
- **Quiz Attempt Deduplication:** Submissions are keyed by immutable UUIDv4 strings. The merge performs an $O(N)$ Set union:
  `mergedAttempts = [...localAttempts, ...remoteAttempts.filter(r => !localAttemptIds.has(r.id))]`

4.2.3 Academic Timeline Promotion Algorithm

Computes a student\'s expected semester automatically from their admission batch year and calendar month:

```
export function calculateCurrentSemester(admissionYear: number): number {
 const now = new Date();
 const currentYear = now.getFullYear();
 const currentMonth = now.getMonth(); // 0 = Jan, 7 = Aug
 // Academic calendar starts in August (Month index 7)
 const elapsedMonths = (currentYear - admissionYear) * 12 + (currentMonth - 7);
 const computedSem = Math.floor(elapsedMonths / 6) + 1;
 return Math.min(8, Math.max(1, computedSem));
}
```

Chapter 5: Real-Time Cross-Device Cloud Sync Engine

5.1 Cloud REST Persistence Architecture

AcademiaQuiz leverages a lightweight REST object persistence architecture:

- **Endpoint:** `https://api.restful-api.dev/objects/ff808181a09d98f701a15976d9952db0`
- **Wildcard CORS Headers:** Returns `Access-Control-Allow-Origin: *`, enabling secure direct client-side requests from `https://sritejamallula.github.io` without proxy servers.
- **JSON Stringification Strategy:** The database object stores internal state in a stringified `data.payload` field. This prevents schema validation errors on deeply nested student and attempt arrays.

5.2 Multi-Trigger Synchronization Lifecycle

Trigger Name	Implementation Code	Operational Impact
Interval Poller	`setInterval(() => syncCloudData(), 8000)`	Continuously polls cloud database every 8 seconds, ensuring all devices reflect recent submissions.
Tab Focus Sync	`window.addEventListener("focus", ...)`	Immediately triggers cloud fetch when a student switches tabs or unlocks a mobile device.
Cross-Tab Broadcast	`new BroadcastChannel("quiz_sync_channel")`	Synchronizes multiple open browser tabs on the same computer in < 5 ms.
Mutation Event	Synchronous call after state mutation	Immediately updates the cloud endpoint upon student registration or quiz submission.

5.3 Zero Dummy Users Verification

The cloud database was formally purged and initialized with clean arrays:

```
GET https://api.restful-api.dev/objects/ff808181a09d98f701a15976d9952db0
Response (200 OK):
{
 "id": "ff808181a09d98f701a15976d9952db0",
 "name": "btech_online_quiz_registry_sritejamallula",
 "data": {
   "payload": "{\\"registeredStudents\\":[],\\"quizAttempts\\":[],\\"lastSyncedAt\\":\\"2026-10-20T18:53:20Z\\"}"
 }
}
```

> **Verification:** No pre-populated students or artificial scores exist. When classmates open the site and register, their roll numbers and names populate the roster in real time.

Chapter 6: Curriculum Mapping (40 Core Subjects)

The application maps directly to the 8-semester B.Tech CSE syllabus under the BVCEC Autonomous R23 pattern:

Sem	Course Code	Theory Subject Title	Key Syllabus & Question Bank Domains

1-1	`BS1101`
	`BS1102`
	`CS1101`
	`ES1101`
		Linear Algebra & Calculus
		Applied Chemistry
		C Programming & Problem Solving
		Basic Civil & Mechanical Engg
			Matrices, Eigenvalues, Cayley-Hamilton, Multiple Integrals
			Water Tech (EDTA, Zeolite), Nernst, Li-ion Batteries
			Control Flow, Pointers, Arrays, Dynamic Memory, Recursion
			Theodolite Surveying, 4-stroke IC Engines, Boilers, TIG Welding

1-2	`BS1201`
	`BS1202`
	`ES1201`
	`CS1201`
	`HS1201`
		Applied Physics
		Mathematics-II (M-2)
		Basic Electrical & Electronics
		Data Structures
		Communicative English
			Thin Film Interference, He-Ne Laser, Superconductivity
			Differential Equations, Laplace Transforms, Vector Calculus
			KCL/KVL, Star-Delta, Transformers, Zener Diodes, BJT
			Stacks, Queues, Linked Lists, Binary Trees, Circular Queues
			Technical Vocabulary, Précis Writing, IEEE Report Formatting

2-1	`CS2101`
	`CS2102`
	`CS2103`
	`CS2104`
	`HS2101`
		Discrete Mathematics (DMGT)
		Database Management Systems
		Java Programming
		Advanced Data Structures
		Universal Human Values
			Propositional Logic, Eulerian Graphs, Recurrences, Posets
			Relational Algebra, SQL NULLs, 3NF/BCNF, B+ Trees, 2PL
			OOP, Multithreading, Exceptions, HashMap Load Factor
			Red-Black Trees, AVL Balance Factor, Bellman-Ford, Master Theorem
			Self-Exploration, Natural Acceptance, Vishwas, 4 Nature Orders

2-2	`CS2205`
	`CS2201`
	`CS2202`
	`CS2203`
	`CS2204`
	`HS2201`
		Python Programming
		Theory of Computation (FLAT)
		Computer Organization (COA)
		Operating Systems
		Software Engineering
		Managerial Economics (MEFA)
			Comprehensions, Decorators, Generators, GIL, Tuples
			DFA/NFA, Pushdown Automata, Pumping Lemma, Turing Machines
			Pipeline Hazards, Cache Mapping, Memory Hierarchy, DMA
			CPU Scheduling (SJF), Banker\'s Deadlock, Virtual Memory, Semaphores
			Agile Scrum, Spiral Model, Black Box Testing, UML Diagrams
			Break-Even Point, Elasticity of Demand, Real Accounts, NPV/IRR

3-1	`23AD5T03`
	`23AD6T05`
	`23AD5D03`
	`23CS5S06`
	`23AD5T02`
		Computer Networks
		Machine Learning
		Exploratory Data Analysis
		Full Stack Development
		Artificial Intelligence
			OSI/TCP-IP, Subnetting /24, 3-Way Handshake, OSPF Dijkstra, BGP
			SVM Kernel Trick, K-Means, Regularization, Precision/Recall
			Outlier Boxplots, Pearson Correlation, Normalization, Chi-Square
			React Virtual DOM, Express Middleware, REST, JWT, WebSockets
			A* Search, Minimax Alpha-Beta Pruning, CSP Heuristics

3-2	`23AD6D06`
	`CS3202`
	`CS3203`
	`CS3204`
	`HS3201`
		Microprocessors & Microcontrollers
		Cryptography & Network Security
		Data Warehousing & Mining
		Cloud Computing
		Professional Ethics & Cyber Laws
			8086 20-bit Address Bus, 8051 Architecture, 8255 PPI
			AES, RSA Factorization, Diffie-Hellman, SHA-256, Firewalls
			OLAP vs OLTP, Star/Snowflake Schema, Apriori Algorithm
			IaaS/PaaS/SaaS, Hypervisors, S3 Storage, Docker, Serverless Lambda
			IT Act 2000 Section 66, Patents/IPR, Whistleblowing, GDPR

4-1	`23AD7D09`
	`23CS7T13`
	`23HM7T07`
	`CS4101`
	`CS4102`
		DevOps
		Deep Learning
		HR & Project Management
		Big Data Analytics
		Information Security & Hacking
			CI/CD, Dockerfiles, Kubernetes Pods, Jenkinsfile, Terraform
			CNN Max Pooling, ReLU, Transformer Self-Attention, LSTM, Adam
			CPM/PERT Critical Path, COCOMO Estimation, Maslow Hierarchy
			Hadoop HDFS NameNode, MapReduce, Spark RDDs, CAP Theorem
			SQL Injection, XSS, Nmap, Burp Suite, CSRF Tokens, ASLR

4-2	`CS4201`
	`CS4202`
	`CS4203`
	`CS4204`
	`CS4205`
		Cyber Security & Forensics
		Distributed Systems
		Software Project Management
		Prompt Engg & Generative AI
		Natural Language Processing
			Chain of Custody, Hardware Write Blockers, Memory Volatility
			Raft/Paxos Consensus, Lamport Clocks, RPC Protocol, 2PC
			Risk Exposure Formula, Agile Burn-Down Charts, CMMI Level 5
			Few-Shot, Chain-of-Thought, RAG Vector Search, LoRA Fine-Tuning
			Stemming/Lemmatization, TF-IDF, Word2Vec, BERT Masked LM

Chapter 7: User Experience & Modules

7.1 Navigation & Registration Interface

The application presents a world-class light glassmorphic theme (`bg-slate-50`, frosted translucent cards, and soft slate typography). The top navigation bar houses:

- **Semester Selector:** Displays active semester (e.g. `Sem 2-1`) and permits seamless browsing across all 8 semesters.
- **Live Cloud Sync Pill:** Real-time status indicator (`■ Live Sync • N Registered`) with manual refresh.
- **Student Profile Badge:** Displays authenticated roll number and name.
- **Faculty Mode Switch:** Toggles between the student evaluation portal and the instructor dashboard.

7.2 Examination Engine

- **10-Minute Countdown Timer:** Real-time countdown clock with color-coded warning pulses when under 2 minutes. Automatically submits upon expiry.
- **10-Question Palette Grid:** Interactive navigation panel showing Answered (Emerald), Flagged (Amber), and Unvisited (Slate) questions.
- **Clutter-Free Quiz Cards:** Clean question presentation with syntax-highlighted code snippets and distraction-free options.

7.3 Score Report & GeeksforGeeks-Style Explanations

- **Performance Metrics:** Displays Final Score (out of 10), Accuracy %, Elapsed Time, and Pass/Fail evaluation.
- **Celebratory Confetti:** Full-screen particle confetti triggers on scores ≥ 80%.
- **Technical Rationale:** Step-by-step pedagogical breakdowns with code samples and theoretical explanations.

7.4 Dynamic 3D Leaderboard & Classmate Roster

- ■ **Score Leaderboard:** Ranks only students who have completed at least one quiz, ordered by total points and accuracy. Top 3 scorers are presented on metallic Gold, Silver, and Bronze 3D podium blocks. Active student is highlighted with a `YOU` badge.
- ■ **All Registered Classmates:** Displays the full roster of registered students across all devices, with branch, admission year, roll number, and registration timestamps.

7.5 Faculty Analytics Portal

- **KPI Metric Cards:** Enrolled Classmates count, Class Average Accuracy %, Overall Pass Rate %, and Active Subjects count.
- **Assessment Controls:** Adjust timer duration, passing cutoff %, negative marking (-0.25 penalty), and instant feedback mode.
- **Submissions Audit Log:** Chronological table of all student quiz attempts.

Chapter 8: Testing, Verification & Results

8.1 Cross-Device Compatibility Verification

Device & Browser	Form Factor	Resolution	Test Outcome
Android Smartphone (Chrome)	Mobile Touch	393 × 873 px	Passed: Responsive question palette, full touch support, fast cloud sync.
Apple iPhone 14 (Safari)	Mobile Touch	390 × 844 px	Passed: Glassmorphism rendered cleanly, timer and confetti animations smooth.
Windows Laptop (Edge/Chrome)	Desktop	1920 × 1080 px	Passed: Full 3D podium rendered, dual-tab leaderboard and faculty portal functional.
MacBook Air (Safari)	Desktop	2560 × 1600 px	Passed: Crisp typography, instant cross-tab broadcast synchronization.

8.2 Real-Time Concurrent Sync Benchmark

Simulated two students registering and taking quizzes simultaneously on different devices:

- **Student 1 (Mobile):** Registered `23A91A0501` and took the DBMS quiz, scoring 9/10.
- **Student 2 (Laptop):** Registered `23A91A0502` and took the Java quiz, scoring 10/10.
- **Result:** Within 8 seconds, both devices reflected both students on the roster and leaderboard without page reloads. Zero dummy accounts were generated.

Chapter 9: Conclusion & Deployment

9.1 Summary of Achievements

The AcademiaQuiz system successfully demonstrates that high-performance, real-time academic evaluation can be delivered through a serverless, decoupled architecture. By eliminating artificial mock data and connecting distributed client devices through an active cloud REST persistence layer, the application provides an authentic, synchronized classroom assessment environment.

9.2 Production Deployment

- **GitHub Repository:** `https://github.com/sritejamallula/online-quiz`
- **Live Application URL:** `https://sritejamallula.github.io/online-quiz/`
- **Documentation Artifact:** `AcademiaQuiz_System_Documentation.pdf`

References & Technical Bibliography

1. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2022). *Introduction to Algorithms* (4th ed.). MIT Press. (Foundational reference for AVL Trees, Red-Black Trees, Bellman-Ford, Floyd-Warshall, and Dynamic Programming algorithms).
2. **JNTU Kakinada (JNTUK)**. (2023). *R23 Course Structure and Syllabi for B.Tech Computer Science & Engineering*. Directorate of Academic & Planning, JNTUK.
3. **BVC Engineering College (Autonomous)**. (2023). *Academic Regulations & Syllabi (Autonomous R23)*. Department of Computer Science & Engineering, BVCEC Odalarevu.
4. **React Documentation Team**. (2024). *React 18 Architecture: Concurrent Mode, Context API, and State Management*. Meta Platforms Inc. Available at: https://react.dev/
5. **Tailwind Labs**. (2024). *Tailwind CSS v4.0 Engine & Modern Glassmorphism Styling*. Available at: https://tailwindcss.com/
6. **Fielding, R. T.** (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine. (REST architectural principles).
7. **W3C Web Platform Working Group**. (2023). *BroadcastChannel API Specification: Cross-Context Message Passing*. World Wide Web Consortium. Available at: https://www.w3.org/TR/webmessaging/
8. **Mozilla Developer Network (MDN)**. (2024). *Client-Side Storage Strategies: LocalStorage, IndexedDB, and Cloud Synchronization*. MDN Web Docs.

END OF PROJECT REPORT
Compiled and Verified for Hard Copy Academic Submission • October 2026